

**ANALISIS TEKSTUAL BERITA BENCANA BANJIR MENGGUNAKAN
PENDEKATAN NATURAL LANGUAGE PROCESSING (NLP)**



Dosen Pengampu :

Irmasari Hafidz, S.Kom., M.Sc.

Assoc. Prof.Dr. Ir. Aris Tjahyanto, M.Kom

Disusun oleh :

Maria Adeline Maharani W.	5026231048
Hafizhan Yusra Sulistyo	5026231060
Zeldano Shan Oeffie	5026231118
Khalila Shafarayhani Atletiko	5026231167
Astrid Meilendra	5026231183
Alisha Rafimalia	5026231202

FAKULTAS TEKNOLOGI ELEKTRO DAN INFORMATIKA CERDAS
INSTITUT TEKNOLOGI SEPULUH NOPEMBER
TAHUN AJARAN 2025-2026

DAFTAR ISI

BAB I	3
PENDAHULUAN	3
1. Latar Belakang	3
2. Rumusan Masalah	4
3. Tujuan Penelitian	4
4. Manfaat Penelitian	5
BAB II	6
TINJAUAN PUSTAKA	6
1. Natural Language Processing (NLP)	6
2. Web Scraping & Akuisisi Data	7
3. Pra-pemrosesan Teks	7
4. Term Frequency-Inverse Document Frequency (TF-IDF)	8
5. POS (Part-of-Speech Tagging)	8
6. NER ((Named Entity Recognition)	9
7. N-Grams	10
8. Sentiment Analysis	10
BAB III	11
METODOLOGI	11
1. Sumber dan Cara Pengumpulan Data	11
2. Tahapan Data Scrapping	11
3. Tahapan Data Preprocessing	11
BAB IV	13
HASIL DAN PEMBAHASAN	13
1. POS (Part-of-Speech Tagging)	13
2. NER ((Named Entity Recognition)	14
3. N-Grams	14
4. TF-IDF (Vectorization)	14
5. Sentiment Analysis Classification	14
BAB V	14
PENUTUP	14
1. Kesimpulan	14
2. Rekomendasi	14
DAFTAR PUSTAKA	14

BAB I

PENDAHULUAN

1. Latar Belakang

Indonesia merupakan negara kepulauan tropis yang secara konsisten menghadapi ancaman bencana hidrometeorologi, terutama banjir. Letak geografis Indonesia yang dilintasi garis khatulistiwa menyebabkan tingginya intensitas curah hujan sepanjang tahun, yang dikombinasikan dengan kondisi topografi berbukit serta kepadatan penduduk di daerah dataran rendah, menjadikan banjir sebagai bencana alam yang paling sering terjadi dan menimbulkan dampak paling luas. Badan Nasional Penanggulangan Bencana (BNPB) mencatat bahwa banjir secara konsisten menduduki peringkat tertinggi dalam frekuensi kejadian bencana di Indonesia setiap tahunnya, dengan dampak mencakup kerugian material, kerusakan infrastruktur, serta gangguan aktivitas sosial-ekonomi masyarakat.

Secara karakteristik, banjir di Indonesia dapat dibedakan menjadi dua kategori utama, yaitu banjir biasa dan banjir bandang. Banjir biasa umumnya terjadi akibat meluapnya air sungai atau drainase yang tidak mampu menampung debit air hujan, sehingga air menggenangi pemukiman warga. Jenis banjir ini umumnya tidak menimbulkan korban jiwa dan bersifat temporer. Sebaliknya, banjir bandang merupakan fenomena yang jauh lebih destruktif, yakni aliran air bah yang membawa material seperti lumpur, batu, dan kayu dengan kecepatan dan volume yang sangat besar. Banjir bandang kerap terjadi di kawasan hulu akibat kombinasi curah hujan ekstrem, longsor, atau jebolnya bendungan alami, serta berpotensi menimbulkan korban jiwa dalam jumlah besar dan kerusakan permanen pada infrastruktur. Perbedaan mendasar antara dua jenis banjir ini memiliki implikasi penting dalam hal respons darurat, prioritas evakuasi, dan kebijakan mitigasi bencana yang harus diterapkan.

Media daring menjadi salah satu sumber informasi paling cepat dan luas dalam menyebarkan laporan kejadian banjir kepada publik maupun pemangku kepentingan. Ribuan artikel berita dari ratusan sumber media online dipublikasikan setiap harinya, mencakup berbagai aspek bencana mulai dari lokasi kejadian, jumlah korban, hingga dampak sosial-ekonomi yang ditimbulkan. Namun, volume data berita yang sangat besar dan bersifat tidak terstruktur menjadi tantangan utama dalam mengekstrak informasi yang bermakna secara efisien dan sistematis. Tanpa pendekatan otomatis yang memadai, proses analisis dan klasifikasi berita banjir akan membutuhkan waktu dan tenaga yang sangat besar, sehingga menghambat kecepatan respons penanggulangan bencana.

Natural Language Processing (NLP) merupakan cabang dari kecerdasan buatan (*Artificial Intelligence*) yang berfokus pada kemampuan komputer untuk

memahami, menginterpretasi, dan memproses bahasa manusia secara otomatis (Khurana et al., 2025). Dalam kerangka yang lebih luas, NLP merupakan bagian integral dari sistem kecerdasan buatan modern yang memungkinkan mesin untuk melakukan penalaran berbasis bahasa, mulai dari klasifikasi teks, ekstraksi entitas, hingga analisis sentimen (Russell & Norvig, 2020). Berbagai teknik NLP seperti *Part-of-Speech (POS) Tagging*, *Named Entity Recognition (NER)*, N-Grams, dan TF-IDF (*Term Frequency-Inverse Document Frequency*) telah terbukti efektif dalam menganalisis pola linguistik, mengidentifikasi kata kunci dominan, dan mengklasifikasikan sentimen dari dokumen teks dalam skala besar (Khurana et al., 2025).

Penelitian ini menerapkan pendekatan NLP untuk menganalisis berita banjir berbahasa Indonesia yang dikumpulkan melalui teknik *web scraping* menggunakan library *newspaper3k* dan BeautifulSoup dari 230 sumber media online. Dataset yang berhasil dikumpulkan berjumlah 2.498 berita, yang dibagi ke dalam dua kategori sentimen: banjir biasa (sentimen positif, tidak menimbulkan korban jiwa) dan banjir bandang (sentimen negatif, menimbulkan korban jiwa). Penerapan NLP pada data berita tidak terstruktur, termasuk melalui proses pra-pemrosesan teks dan ekstraksi fitur, telah terbukti mampu menghasilkan wawasan yang bermakna dan mendukung pengambilan keputusan berbasis data (Samuel & Prasetya Kristiadi, 2025). Dengan demikian, penelitian ini diharapkan dapat memberikan kontribusi nyata bagi pengembangan sistem klasifikasi berita bencana secara otomatis serta mendukung upaya mitigasi dan penanggulangan banjir di Indonesia.

2. Rumusan Masalah

Berdasarkan latar belakang yang telah diuraikan, maka rumusan masalah dalam penelitian ini adalah sebagai berikut:

1. Bagaimana proses pengumpulan dan pra-pemrosesan data berita banjir dari berbagai sumber media online Indonesia menggunakan teknik *web scraping* dan metode NLP?
2. Bagaimana penerapan metode analisis tekstual NLP, meliputi POS Tagging, Named Entity Recognition (NER), N-Grams, dan TF-IDF, dalam membedakan karakteristik linguistik berita banjir biasa dan banjir bandang?
3. Bagaimana representasi vektor teks dapat dimanfaatkan untuk membangun model klasifikasi sentimen otomatis antara berita banjir biasa (positif) dan banjir bandang (negatif)?

3. Tujuan Penelitian

Berdasarkan rumusan masalah di atas, tujuan penelitian ini adalah sebagai berikut:

1. Mengumpulkan dan membangun dataset berita banjir berbahasa Indonesia melalui teknik *web scraping*, serta melakukan pra-pemrosesan teks menggunakan metode NLP termasuk lemmatisasi berbasis Sastrawi.

2. Menerapkan dan menganalisis metode POS Tagging, NER, N-Grams, dan TF-IDF untuk mengidentifikasi pola linguistik dan kata kunci yang membedakan berita banjir biasa dari banjir bandang.
3. Membangun representasi vektor teks sebagai dasar klasifikasi sentimen otomatis untuk membedakan kategori banjir biasa (sentimen positif) dan banjir bandang (sentimen negatif).

4. Manfaat Penelitian

Penelitian ini diharapkan memberikan manfaat sebagai berikut:

A. Manfaat Teoritis

1. Memberikan kontribusi akademis pada pengembangan metode analisis teks berbahasa Indonesia menggunakan NLP, khususnya pada domain berita bencana alam, sebagaimana didukung oleh kajian literatur terkini (Khurana et al., 2025).
2. Memperkaya literatur ilmiah mengenai pemanfaatan teknik NLP POS Tagging, NER, N-Grams, dan TF-IDF dalam konteks analisis sentimen berita berbahasa Indonesia.

B. Manfaat Praktis

1. Bagi pemerintah dan lembaga penanggulangan bencana: menyediakan dasar pengembangan sistem pemantauan berita bencana otomatis untuk mendukung respons yang lebih cepat dan tepat sasaran.
2. Bagi masyarakat: membantu penyebaran informasi yang lebih terklasifikasi mengenai jenis banjir, sehingga meningkatkan kewaspadaan dan kesiapsiagaan publik.
3. Bagi peneliti dan akademisi: menyediakan dataset berita banjir berbahasa Indonesia yang telah dianotasi untuk keperluan penelitian NLP lebih lanjut (Samuel & Prasetya Kristiadi, 2025).
4. Bagi pengembang sistem: memberikan landasan teknis dan metodologis untuk aplikasi cerdas berbasis NLP dalam klasifikasi dan peringkasan berita bencana secara otomatis.

BAB II

TINJAUAN PUSTAKA

1. Natural Language Processing (NLP)

Natural Language Processing (NLP) merupakan cabang dari kecerdasan buatan (*Artificial Intelligence*) yang berfokus pada kemampuan komputer untuk memahami, mengolah, dan menganalisis bahasa manusia secara otomatis. NLP memungkinkan komputer untuk memproses data teks dalam jumlah besar serta mengekstraksi informasi yang bermakna dari bahasa alami, baik dalam bentuk tulisan maupun lisan (Khurana et al., 2025).

Dalam perkembangannya, NLP menjadi salah satu bidang penting dalam teknologi modern karena banyak digunakan dalam berbagai aplikasi, seperti mesin pencari, chatbot, sistem rekomendasi, hingga analisis sentimen. NLP bekerja dengan menggabungkan konsep linguistik dan teknik komputasi untuk memahami struktur dan makna suatu teks (Russell & Norvig, 2020).

Secara umum, proses dalam NLP melibatkan beberapa tahapan utama, yaitu:

- **Akuisisi Data Teks**
Mengumpulkan data dalam bentuk teks dari berbagai sumber, seperti website, media sosial, atau dokumen digital.
- **Pra-pemrosesan Teks**
Membersihkan dan menormalisasi data teks agar dapat diproses oleh mesin, misalnya melalui tokenisasi, case folding, dan penghapusan *stopwords*.
- **Ekstraksi Fitur**
Mengubah teks menjadi representasi numerik, seperti menggunakan metode TF-IDF atau N-Grams.
- **Analisis dan Pemodelan**
Menggunakan algoritma tertentu untuk melakukan klasifikasi, clustering, atau analisis sentimen terhadap data teks.

Dalam penelitian ini, NLP digunakan untuk menganalisis berita banjir berbahasa Indonesia yang diperoleh dari berbagai sumber media daring. Teknik NLP seperti pra-pemrosesan teks, POS Tagging, NER, N-Grams, dan TF-IDF diterapkan untuk mengekstraksi informasi penting serta mengidentifikasi pola linguistik dalam teks. Hasil analisis ini kemudian digunakan untuk mendukung proses klasifikasi berita banjir menjadi dua kategori, yaitu banjir biasa dan banjir bandang.

2. Web Scraping & Akuisisi Data

Web scraping merupakan teknik pengumpulan data secara otomatis dari website dengan cara mengekstrak informasi yang terdapat pada halaman web. Teknik ini digunakan untuk memperoleh data dalam jumlah besar dari berbagai sumber secara efisien tanpa harus melakukan pengumpulan secara manual.

Dalam proses web scraping, sistem akan mengakses halaman web, mengambil konten yang dibutuhkan, kemudian mengekstraknya ke dalam format yang terstruktur seperti tabel atau dataset. Proses ini umumnya dilakukan menggunakan pustaka pemrograman seperti BeautifulSoup dan newspaper3k, yang memungkinkan pengambilan elemen teks seperti judul berita, isi artikel, dan metadata lainnya.

Akuisisi data merupakan tahap awal dalam penelitian yang bertujuan untuk mengumpulkan data yang relevan dengan topik yang diteliti. Kualitas data yang diperoleh sangat mempengaruhi hasil analisis yang akan dilakukan pada tahap selanjutnya.

Pada penelitian ini, proses web scraping digunakan untuk mengumpulkan berita banjir dari berbagai media daring di Indonesia. Data yang diperoleh kemudian disimpan dalam bentuk dataset dan digunakan sebagai bahan utama dalam proses analisis menggunakan metode NLP.

3. Pra-pemrosesan Teks

Pra-pemrosesan dilakukan setelah content-scraping, dengan tujuan untuk menormalisasi teks sesuai dengan format yang dapat dipahami oleh mesin (*machine-readable format*). Beberapa tahap normalisasi yang perlu dilakukan seperti :

1. Konsistensi Struktur Data

Teks mentah hasil scraping yang berantakan, akan diubah menjadi huruf kecil (case folding), sehingga kata “Data”, “DATA”, dan “data” dianggap sebagai satu entitas yang sama oleh algoritma.

2. Tokenization

Data teks yang sudah di normalisasi akan dipecah menjadi sekumpulan karakter atau kalimat menjadi unit-unit yang lebih kecil dan bermakna yang disebut token. Token ini biasanya berupa kata, frasa, atau sub-kata, dengan tujuan untuk menyederhanakan struktur teks sehingga mesin dapat mengidentifikasi pola frekuensi kata dan hubungan antarkata dengan lebih mudah.

3. Standar linguistik dan Semantik

Teks dinormalisasikan agar kembali ke bentuk dasarnya melalui proses:

- a. Lemmatization/Stemming : Menghapus imbuhan untuk mendapat kata dasar (misal: “memproses” menjadi “proses”)

- b. Slang/Formalization : Mengubah bahasa gaul atau singkatan menjadi kata baku agar sesuai dengan kamus referensi yang digunakan.

4. Reduksi Derau (Noise Reduction)

Menghapus elemen yang tidak memiliki nilai informatif dalam analisis teks, seperti :

- a. Tanda baca, angka, dan karakter khusus.
- b. Stopwords (kata umum seperti “dan”, “yang”, “di”) yang sering muncul tetapi tidak membawa makna unik bagi model.

4. Term Frequency-Inverse Document Frequency (TF-IDF)

Term Frequency-Inverse Document Frequency (TF-IDF) adalah metode pembobotan kata dalam Information Retrieval dan Natural Language Processing yang diperkenalkan oleh Salton & Buckley (1988). Ide utamanya adalah memberikan bobot tinggi pada kata yang sering muncul di suatu dokumen tetapi jarang muncul di dokumen lain, sehingga kata tersebut dianggap sebagai penciri penting dari dokumen tersebut.

TF-IDF terdiri dari dua komponen utama. Komponen pertama adalah *Term Frequency* (TF) yang mengukur seberapa sering sebuah kata muncul dalam satu dokumen. Semakin sering suatu kata muncul, semakin besar bobotnya terhadap dokumen tersebut. Komponen kedua adalah *Inverse Document Frequency* (IDF) yang berfungsi mengurangi bobot kata-kata yang terlalu umum dan muncul di hampir semua dokumen sehingga tidak cukup informatif untuk membedakan antar dokumen. Skor TF-IDF akhir diperoleh dengan mengalikan kedua nilai tersebut, lalu dinormalisasi agar dokumen dengan panjang berbeda tetap dapat dibandingkan secara adil.

TF-IDF diimplementasikan menggunakan *TfidfVectorizer* dari pustaka *scikit-learn* Python. Selain unigram (kata tunggal), dapat digunakan pula bigram yaitu pasangan dua kata berurutan seperti "banjir bandang" atau "korban jiwa" agar frasa yang bermakna dapat ditangkap sebagai satu kesatuan fitur. Hasil representasi TF-IDF berupa matriks numerik yang dapat digunakan sebagai input untuk berbagai algoritma machine learning, seperti klasifikasi teks, clustering dokumen, maupun analisis sentimen.

5. POS (Part-of-Speech Tagging)

Part-of-Speech Tagging (POS Tagging) adalah teknik dalam Natural Language Processing (NLP) yang digunakan untuk mengidentifikasi dan memberikan label kelas kata pada setiap token dalam sebuah teks. Kelas kata tersebut dapat berupa kata benda (*noun*), kata kerja (*verb*), kata sifat (*adjective*), kata keterangan (*adverb*), dan kategori lainnya. Proses ini membantu komputer memahami struktur gramatikal suatu kalimat sehingga teks dapat dianalisis secara lebih sistematis.

Dalam penerapannya, POS Tagging umumnya dilakukan setelah tahap pra-pemrosesan teks, seperti tokenisasi dan lemmatisasi. Teks yang telah diproses kemudian dianalisis untuk menentukan kategori gramatikal dari setiap kata. Hasil POS Tagging biasanya ditampilkan dalam bentuk pasangan antara kata dan label kelas katanya, misalnya kata “banjir” diberi label *noun*, sedangkan kata “melanda” diberi label *verb*.

Secara umum, POS Tagging dapat dilakukan melalui beberapa pendekatan, yaitu:

- **Berbasis Aturan (*Rule-based*)**
Pendekatan ini menggunakan aturan tata bahasa yang telah ditentukan sebelumnya untuk menentukan kelas kata dari suatu kata.
- **Berbasis Statistik**
Pendekatan ini menentukan kelas kata berdasarkan kemungkinan atau probabilitas kemunculan suatu kata dalam konteks tertentu.
- **Pembelajaran Mesin dan *Deep Learning***
Pendekatan modern ini menggunakan model yang dilatih pada data berlabel untuk mengenali pola bahasa secara otomatis. Salah satu pustaka yang dapat digunakan untuk POS Tagging adalah Stanza, yaitu perangkat NLP yang menyediakan model bahasa untuk berbagai bahasa, termasuk bahasa Indonesia.

Pada penelitian ini, POS Tagging dilakukan menggunakan pustaka Stanza dengan model bahasa Indonesia. Proses POS Tagging diterapkan pada kolom hasil lemmatisasi (*After_lemma*) dari dua dataset, yaitu berita banjir biasa dan berita banjir bandang. Setiap kata dalam teks diberi label kelas kata menggunakan tag UPOS (Universal Part-of-Speech), kemudian hasilnya disimpan dalam kolom POS_Tags. Hasil POS Tagging ini digunakan untuk melihat pola penggunaan kelas kata pada masing-masing kategori berita, sehingga dapat membantu membedakan karakteristik linguistik antara berita banjir biasa dan banjir bandang.

6. NER ((Named Entity Recognition))

Named Entity Recognition (NER) adalah sebuah teknik ekstraksi informasi yang dirancang untuk mengenali dan mengkategorikan kata atau frasa penting (entitas) di dalam sebuah teks ke dalam kelompok yang sudah ditentukan sebelumnya. Dalam teks umum, kategori ini biasanya mencakup nama orang, nama organisasi, lokasi geografis, dan keterangan waktu. Secara historis, sistem NER berkembang dari pendekatan yang sangat sederhana hingga menggunakan kecerdasan buatan:

1. **Berbasis Aturan dan Kamus (Rule-based/Lexicon):** Model mencari kata berdasarkan daftar kamus yang sudah ada atau menggunakan pola tata bahasa

baku. Pendekatan ini akurat namun kaku karena sistem tidak bisa mengenali kata baru yang tidak ada di dalam kamusnya.

2. Pembelajaran Mesin Tradisional: Menggunakan model statistik yang mempelajari probabilitas urutan kata dari data yang sudah diberi label oleh manusia.
3. Pembelajaran Mendalam (Deep Learning): Pendekatan modern yang menggunakan jaringan saraf tiruan terpadu. Model ini mampu mempelajari pola bahasa dan konteks kalimat secara otomatis dari jutaan teks tanpa perlu aturan manual, menjadikannya standar industri saat ini.

7. N-Grams

Jika NER berfokus pada pengenalan entitas spesifik, N-grams berfokus pada pemahaman konteks. Dalam NLP, teks biasanya dipecah menjadi unit-unit kecil yang disebut token (biasanya berupa kata). N-grams adalah gabungan dari beberapa token yang letaknya berdekatan di dalam sebuah kalimat. Jenis N-grams dikategorikan berdasarkan jumlah kata yang digabungkan:

1. Unigram: Satu kata mandiri (misal: "sangat").
2. Bigram: Kombinasi dua kata berurutan (misal: "sangat baik").
3. Trigram: Kombinasi tiga kata berurutan (misal: "sangat baik sekali").

Tujuan utama penggunaan fitur N-grams adalah agar komputer tidak kehilangan konteks kalimat. Sebagai contoh, jika komputer hanya membaca kata per kata (unigram), kata "tidak" dan "baik" akan diartikan secara terpisah. Dengan menggunakan bigram ("tidak baik"), mesin dapat langsung menangkap bahwa frasa tersebut memiliki makna yang berlawanan dari sekadar kata "baik"

8. Sentiment Analysis

Analisis sentimen (Sentiment Analysis) adalah salah satu cabang penting dari Natural Language Processing (NLP) yang bertujuan untuk mengekstraksi, mengidentifikasi, dan mengklasifikasikan informasi subjektif dari teks. Secara umum, analisis sentimen bertujuan untuk menentukan polaritas emosional dari suatu teks, yaitu apakah teks tersebut bersentimen positif, negatif, atau netral.

Dalam konteks analisis berita bencana alam, seperti banjir, analisis sentimen memiliki peran krusial. Berita mengenai bencana dapat diklasifikasikan tidak hanya berdasarkan opini, tetapi juga berdasarkan tingkat keparahan atau dampaknya. Misalnya, berita yang melaporkan banjir dengan korban jiwa, kerusakan infrastruktur yang parah (seperti banjir bandang), dapat dikategorikan bersentimen negatif. Sebaliknya, berita mengenai banjir yang bersifat temporer tanpa korban jiwa (banjir biasa) dapat dikategorikan bersentimen netral atau positif relatif terhadap keparahan bencana lainnya.

Terdapat beberapa pendekatan utama dalam melakukan analisis sentimen:

1. Pendekatan Berbasis Aturan atau Kamus (Lexicon-based / Keyword-based): Pendekatan ini mengklasifikasikan sentimen berdasarkan kehadiran kata-kata

kunci tertentu yang telah ditentukan sebelumnya. Sebuah teks akan dievaluasi berdasarkan skor dari kata-kata kunci tersebut. Jika skor kata-kata negatif lebih tinggi, maka teks diklasifikasikan sebagai negatif, dan sebaliknya. Keuntungan metode ini adalah tidak memerlukan proses pelatihan (training) model. Namun, kelemahannya adalah metode ini kaku dan kesulitan memahami konteks kalimat jika kata kunci tidak muncul.

2. Pendekatan Pembelajaran Mesin (Machine Learning): Pendekatan ini menggunakan algoritma pembelajaran mesin untuk melatih model klasifikasi pada dataset yang telah diberi label (ground truth). Model ini belajar mengenali pola dan kombinasi kata yang berkaitan dengan sentimen tertentu. Algoritma seperti Naive Bayes sering digunakan karena efisien dan efektif untuk klasifikasi teks. Model ini biasanya menggunakan representasi numerik teks seperti TF-IDF sebagai fitur masukan. Pendekatan ini umumnya menghasilkan akurasi yang lebih baik daripada pendekatan berbasis kata kunci karena kemampuannya menangkap pola linguistik yang lebih kompleks.

BAB III

METODOLOGI

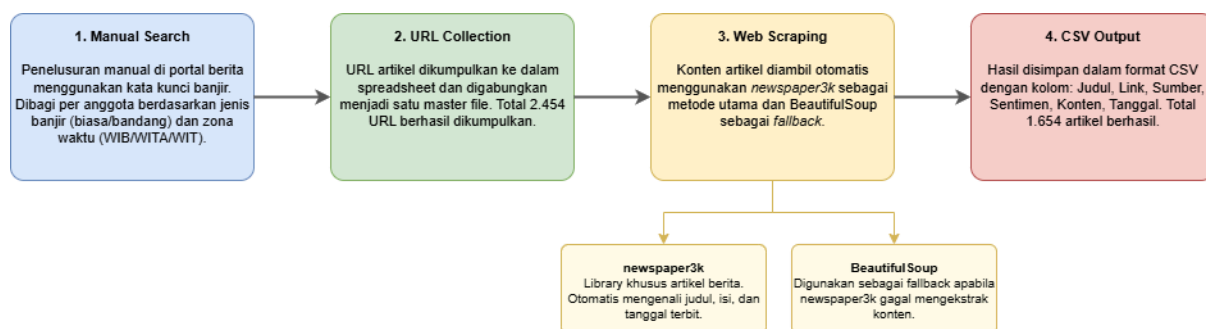
1. Sumber dan Cara Pengumpulan Data

Data yang digunakan dalam penelitian ini merupakan data sekunder yang bersumber dari artikel berita daring yang dipublikasikan oleh berbagai portal media terpercaya. Proses pengumpulan data dilakukan secara sistematis dengan menelusuri tautan (link) berita yang relevan dengan topik bencana banjir. Untuk memastikan data yang terkumpul memiliki cakupan spasial yang luas dan representatif, pencarian tautan dilakukan dengan membagi fokus pengumpulan ke dalam beberapa daerah terdampak yang telah ditentukan sebelumnya.

Dalam tahap pelaksanaannya, masing-masing anggota mengumpulkan sebanyak 400 tautan berita untuk masing-masing daerah tersebut, guna menjamin ketersediaan volume data yang cukup besar untuk diolah pada tahap analisis. Seluruh tautan yang telah berhasil dikumpulkan kemudian dihimpun secara terpusat ke dalam satu lembar kerja (spreadsheet) untuk memudahkan proses verifikasi dan manajemen data. Sebagai tahap akhir dari pengumpulan data, dokumen spreadsheet tersebut dikonversi dan disimpan ke dalam format Comma-Separated Values (CSV). Penggunaan format CSV ini bertujuan untuk memastikan bahwa data dapat diintegrasikan dengan mudah ke dalam lingkungan pemrograman Python, sehingga proses ekstraksi informasi menggunakan teknik Natural Language Processing (NLP) dapat berjalan secara optimal.

2. Tahapan Data Scrapping

Setelah seluruh URL artikel dikumpulkan ke dalam satu file spreadsheet dan digabungkan menjadi master data, proses scraping otomatis dijalankan menggunakan Python di lingkungan Google Colab. Alur keseluruhan tahapan scraping dapat dilihat pada Gambar 3.2.



Gambar 3.2 Alur pengumpulan data berita banjir

Tahapan scraping dilakukan sebagai berikut. Pertama, data URL dibaca ke dalam dataframe menggunakan `pandas` dan dilakukan pembersihan awal berupa penghapusan baris duplikat serta baris yang tidak memiliki URL valid. Pada tahap ini juga dilakukan identifikasi sumber portal secara otomatis berdasarkan domain URL menggunakan fungsi deteksi yang memetakan domain ke nama portal yang sesuai.

Kedua, untuk setiap URL dalam dataframe, program mencoba mengekstrak konten artikel menggunakan library `newspaper3k`. Apabila konten yang berhasil diambil memenuhi batas minimum panjang teks, hasil tersebut langsung disimpan. Apabila `newspaper3k` gagal, program

secara otomatis beralih ke BeautifulSoup sebagai fallback dengan mengambil seluruh elemen paragraf dari halaman HTML. Fungsi utama yang menangani logika ini ditampilkan pada Gambar 3.3.

```
def scrape_artikel(url, max_retries=2):
    """coba scrape dengan newspaper dulu, fallback ke bs4"""
    for attempt in range(max_retries):
        try:
            # coba newspaper3k dulu
            result = scrape_newspaper(url)
            if result:
                return result
        except:
            pass

        try:
            # fallback ke bs4
            result = scrape_bs4(url)
            if result:
                return result
        except:
            pass

        # tunggu sebentar sebelum retry
        if attempt < max_retries - 1:
            time.sleep(2)

    return None
```

Gambar 3.3 Fungsi scraping dengan mekanisme fallback BeautifulSoup

Ketiga, untuk mencegah server portal berita memblokir permintaan akibat frekuensi akses yang terlalu tinggi, diberikan jeda waktu antar setiap permintaan. Selain itu, diterapkan sistem checkpoint yang menyimpan progress scraping secara berkala ke dalam file CSV sementara, sehingga apabila terjadi gangguan koneksi atau sesi Colab terputus, proses dapat dilanjutkan dari titik terakhir tanpa mengulang dari awal.

Dari total 2.454 artikel yang dicoba untuk di-scrape, sebanyak 1.654 artikel berhasil diekstrak kontennya. Sebanyak 800 artikel tidak berhasil diambil kontennya karena berbagai alasan, antara lain artikel telah dihapus dari portal, konten berada di balik sistem berlangganan berbayar, struktur HTML halaman tidak standar, atau server memblokir permintaan otomatis. Hasil akhir scraping disimpan dalam format CSV dengan kolom Judul, Link, Sumber, Sentimen, Konten, dan Tanggal seperti yang terlihat pada Gambar 3.4.

No	Judul	Link	Sumber	Sentimen	Konten	Tanggal
1	Akses Jalan Perumnas Antang Lumpuh Akibat Banjir...	https://www.detik.com/sulsel/makassar/d-837182...	Detik	Banjir	Kawasan Perumnas Antang, Makassar, Sulawe... Kota	2026-02-25 11:43:23

2	Banjir di Makassar, 545 Warga Pengungsi Dievak...	https://news.detik.com/berita/d-8371585/banjir...	Detik	Banjir	Badan Penanggulangan Bencana Daerah (BPBD) Mak...	2026-02-25 09:42:47
3	Motor Mogok di Jalan Inspeksi Kanal Borong Mak...	https://www.detik.com/sulsel/makassar/d-837154...	Detik	Banjir	Waduk Tunggu Pampang, Kota Makassar, Sulawesi ...	2026-02-25 09:13:22

Gambar 3.4 Struktur dataset hasil scraping dalam format CSV

3. Tahapan Data Preprocessing

3.1. Normalisasi dan Punctuation Removal

Tahap pertama dalam proses preprocessing adalah normalisasi teks. Normalisasi bertujuan untuk menyeragamkan bentuk teks agar tidak terdapat inkonsistensi yang dapat mempengaruhi hasil analisis. Proses ini mencakup konversi seluruh karakter huruf menjadi huruf kecil (*lowercase*), sehingga kata seperti "Data" dan "data" diperlakukan sebagai token yang sama.

Selain normalisasi huruf, tahap ini juga melibatkan penghapusan tanda baca (*punctuation removal*). Tanda baca seperti titik (.), koma (,), tanda seru (!), tanda tanya (?), dan karakter khusus lainnya dihilangkan dari teks karena umumnya tidak membawa makna semantik yang relevan dalam analisis teks. Proses ini dilakukan menggunakan ekspresi reguler (*regular expression*) yang secara sistematis mendeteksi dan menghapus karakter non-alfanumerik dari setiap token.

3.2. Noise Reduction (Stop Words)

Tahap kedua adalah pengurangan noise melalui penghapusan *stop words*. Proses penghapusan stop words dilakukan dengan mencocokkan setiap token hasil normalisasi terhadap daftar stop words yang telah ditentukan sebelumnya (*stop word list*). Daftar ini dapat bersumber dari library NLP standar seperti NLTK atau Sastrawi untuk Bahasa Indonesia, dan dapat disesuaikan (*customized*) sesuai dengan domain atau konteks penelitian yang sedang dilakukan. Token yang ditemukan dalam daftar stop words akan dieliminasi dari teks.

Dengan dihapusnya stop words, ukuran kosakata (*vocabulary size*) berkurang secara signifikan sehingga model dapat lebih berfokus pada kata-kata bermakna (*content words*) yang benar-benar merepresentasikan topik dan isi dari dokumen. Hal ini pada akhirnya meningkatkan efisiensi komputasi sekaligus kualitas fitur yang dihasilkan untuk proses analisis selanjutnya.

3.3. Lemmatization

Tahap selanjutnya dalam proses preprocessing adalah *lemmatization*. Lemmatization merupakan proses mengubah kata ke bentuk dasarnya (*lemma*) dengan

mempertimbangkan struktur morfologi dan konteks linguistik. Berbeda dengan *stemming* yang hanya memotong imbuhan secara sederhana, lemmatization menghasilkan kata dasar yang lebih akurat dan sesuai dengan kaidah bahasa.

Sebagai contoh, kata “memproses”, “diproses”, dan “pemrosesan” akan dikonversi menjadi bentuk dasar yang sama, yaitu “proses”. Proses ini bertujuan untuk mengurangi variasi kata yang memiliki makna serupa sehingga dapat meningkatkan konsistensi data dalam analisis teks.

Dalam penelitian ini, proses lemmatization dilakukan menggunakan pustaka NLP bahasa Indonesia, seperti Sastrawi, yang mampu menangani imbuhan dalam Bahasa Indonesia secara lebih efektif. Hasil dari proses ini disimpan dalam kolom *After_lemma*, yang kemudian digunakan sebagai input utama dalam tahap analisis selanjutnya, seperti POS Tagging dan ekstraksi fitur.

Dengan adanya lemmatization, jumlah variasi kata dalam dataset dapat diminimalkan, sehingga model lebih mudah dalam mengenali pola dan hubungan antar kata. Hal ini juga berkontribusi pada peningkatan akurasi dalam proses klasifikasi teks.

BAB IV

HASIL DAN PEMBAHASAN

1. POS (Part-of-Speech Tagging)

1.1. Persiapan dan Proses POS Tagging

Pada tahap ini dilakukan proses pelabelan kelas kata menggunakan metode POS Tagging dengan bantuan library Stanza. Dataset yang digunakan merupakan data hasil preprocessing dengan fokus pada kolom `After_lemma` yang berisi teks yang telah dinormalisasi.

```
# 5. Fungsi POS Tagging Stanza
def pos_tagging_stanza(text):
    if isinstance(text, str) and text.strip() != "":
        # Memproses teks
        doc = nlp_stanza(text)
        result = []
        for sentence in doc.sentences:
            for word in sentence.words:
                result.append((word.text, word.upos))
        return result
    return []

# 6. Eksekusi POS Tagging
print("Memproses POS Tagging untuk data banjir biasa...")
df_sentimen_banjir['POS_Tags'] = df_sentimen_banjir['After_lemma'].progress_apply(pos_tagging_stanza)

print("Memproses POS Tagging untuk data banjir bandang...")
df_sentimen_bandang['POS_Tags'] = df_sentimen_bandang['After_lemma'].progress_apply(pos_tagging_stanza)

# 7. Simpan Hasil ke CSV Baru
print("Menyimpan hasil...")
df_sentimen_banjir.to_csv('hasil_pos_banjir_biasa.csv', index=False)
df_sentimen_bandang.to_csv('hasil_pos_banjir_bandang.csv', index=False)

print("\nProses selesai! File 'hasil_pos_banjir_biasa.csv' dan 'hasil_pos_banjir_bandang.csv' sudah tersimpan.")
display(df_sentimen_banjir[['After_lemma', 'POS_Tags']].head(3))
```

Model bahasa Indonesia diunduh menggunakan `stanza.download('id')`, kemudian dibuat pipeline dengan processor `tokenize` dan `pos`. Processor `tokenize` berfungsi untuk memecah teks menjadi token atau kata, sedangkan `pos` digunakan untuk menentukan kelas kata dari setiap token.

Setiap teks diproses menggunakan fungsi `pos_tagging_stanza()` yang menghasilkan pasangan kata dan label POS seperti NOUN, VERB, ADJ, PROPN, dan lainnya. Proses ini diterapkan ke seluruh dataset menggunakan `progress_apply()` sehingga proses dapat dipantau.

Proses selesai! File 'hasil_pos_banjir_biasa.csv' dan 'hasil_pos_banjir_bandang.csv' sudah tersimpan.

	After_lemma	POS_Tags
0	wali kota makasar munafri api arifudin tetap s...	[(wali, NOUN), (kota, NOUN), (makasar, NOUN), ...
1	jumlah ruas jalan kota makasar sulawesi selata...	[(jumlah, NOUN), (ruas, NOUN), (jalan, NOUN), ...
2	badan penanggulangan bencana daerah bpbd makasa...	[(badan, NOUN), (penanggulangan, NOUN), (bencan...

Output ini menunjukkan bahwa proses POS Tagging telah berhasil dilakukan, di mana setiap dokumen tidak hanya direpresentasikan sebagai teks, tetapi juga sebagai struktur linguistik yang memuat informasi kelas kata. Hasil ini selanjutnya dapat digunakan untuk analisis lanjutan atau sebagai fitur tambahan dalam proses machine learning.

1.2. Pembentukan dan Penyimpanan Data POS Tagging

Setelah proses tagging selesai, hasilnya disimpan ke dalam kolom baru bernama POS_Tags. Kolom ini berisi daftar pasangan kata dan label kelas katanya untuk setiap dokumen.

```
import ast
from IPython.display import HTML, display

def visualize_pos_tags_highlight(pos_data, max_tokens=150):
    if isinstance(pos_data, str):
        try:
            pos_list = ast.literal_eval(pos_data)
        except:
            return "<p>Gagal membaca format data.</p>"
    else:
        pos_list = pos_data

    if not isinstance(pos_list, list):
        return "<p>Data tidak tersedia.</p>"

    pos_list = pos_list[:max_tokens]

    # Mapping warna LENGKAP (Mendukung format nlp-id maupun format Stanza/Universal)
    color_map = {
        # Kata Benda (Hijau / Biru Muda)
        'NN': '#e2f0cb', 'NOUN': '#e2f0cb', 'NND': '#e2f0cb',
        'NNP': '#cce3f6', 'PROPN': '#cce3f6',
        # Kata Kerja (Pink)
        'VB': '#ffdfd3', 'VERB': '#ffdfd3',
        # Kata Sifat (Kuning)
        'JJ': '#fdfd96', 'ADJ': '#fdfd96',
        # Kata Keterangan (Cyan)
        'RB': '#c5eae5', 'ADV': '#c5eae5',
        # Angka (Oranye)
        'CD': '#ffb347', 'OD': '#ffb347', 'NUM': '#ffb347',
        # Kata Ganti (Ungu Muda)
        'PR': '#e2d9f3', 'PRP': '#e2d9f3', 'PRON': '#e2d9f3',
        # Kata Depan / Hubung / Lain-lain (Abu-abu muda)
        'IN': '#f0f0f0', 'ADP': '#f0f0f0',
        'CC': '#f0f0f0', 'SC': '#f0f0f0', 'CCONJ': '#f0f0f0', 'SCONJ': '#f0f0f0'
    }

    html_content = ""
    for word, tag in pos_list:
        # Hapus spasi dan jadikan huruf besar agar cocok dengan color_map
        clean_tag = tag.strip().upper()
        bg_color = color_map.get(clean_tag, '#ffffff')

        html_content += f"""
        <span style="background-color: {bg_color}; border: 1px solid #bbb; border-radius: 4px; padding: 2px 6px; margin-right: 10px;">
            <span style="font-weight: bold; color: #000; font-size: 14px;">{word}</span>
            <span style="font-size: 10px; font-weight: 800; color: #444; margin-left: 3px;">[{clean_tag}]</span>
        </span>
        """
    return html_content
```

Hasil POS Tagging kemudian disimpan ke dalam file CSV baru, yaitu `hasil_pos_banjir_biasa.csv` dan `hasil_pos_banjir_bandang.csv`. Selain itu, ditampilkan sebagian data dalam bentuk tabel untuk memastikan bahwa proses tagging telah berjalan dengan baik. Struktur data ini memungkinkan hasil POS

Tagging digunakan kembali untuk analisis lanjutan tanpa perlu melakukan proses ulang.

1.3. Visualisasi POS Tagging

Pada tahap ini dilakukan visualisasi hasil POS Tagging untuk mempermudah interpretasi. Visualisasi dibuat menggunakan HTML, di mana setiap kata ditampilkan dalam bentuk label berwarna sesuai dengan kelas katanya.

```
# Select one row to visualize
sample_index = 0

# Menggunakan dataframe 'df_sentimen_banjir' dan kolom 'POS_Tags'
html_pos = visualize_pos_tags_highlight(
    df_sentimen_banjir.loc[sample_index, 'POS_Tags'],
    max_tokens=150
)

display(HTML(f"""
<h3>POS Tagging Visualization</h3>
<div style="line-height:2.1; font-size:15px; padding:10px; background-color:#2b2b2b; border-radius:8px;">
{html_pos}
</div>
"""))
```

POS Tagging Visualization



Warna yang digunakan berbeda untuk setiap POS Tag, seperti hijau untuk NOUN, merah muda untuk VERB, kuning untuk ADJ, dan biru muda untuk PROPN. Intensitas dan variasi warna membantu dalam mengidentifikasi distribusi kelas kata dalam teks.

Visualisasi ini memberikan gambaran yang lebih intuitif terhadap struktur linguistik dalam dokumen, sehingga memudahkan analisis terhadap pola penggunaan kata pada data banjir biasa dan banjir bandang.

2. NER ((Named Entity Recognition))

2.1 Implementasi dan Arsitektur Kode

Untuk menjalankan proses ini, digunakan model deep learning IndoBERT (cahya/bert-base-indonesian-NER) yang telah dilatih pada korpus bahasa Indonesia sehingga memiliki sensitivitas yang tinggi terhadap nama-nama wilayah lokal. Secara teknis, model ini diimplementasikan menggunakan pipeline dari pustaka transformers. Salah satu tantangan terbesar dalam penggunaan model berbasis transformer adalah batasan panjang input yang hanya mampu memproses maksimal 512 token.

```

# 4. Inisialisasi Model IndoBERT
print("Memuat model NER IndoBERT...")
model_name = "cahya/bert-base-indonesian-NER"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForTokenClassification.from_pretrained(model_name)

ner_pipeline = pipeline("ner", model=model, tokenizer=tokenizer, aggregation_strategy="simple")

# 5. Fungsi Ekstraksi NER dengan Chunking
# (Sangat penting untuk menghindari error batas maksimal token dari model BERT)
def extract_entities_chunked(text, chunk_size=300):
    chunks = [text[i:i+chunk_size] for i in range(0, len(text), chunk_size)]
    results = []

    for chunk in chunks:
        try:
            ents = ner_pipeline(chunk)
            results.extend([(ent['word'], ent['entity_group']) for ent in ents])
        except Exception as e:
            continue

    return results

```

Untuk mengatasi kendala tersebut, dikembangkan sebuah fungsi khusus bernama `extract_entities_chunked`. Logika utama dari kode ini adalah membagi naskah berita yang panjang menjadi beberapa potongan (chunks) kecil. Setiap potongan kemudian diproses secara terpisah oleh model, dan hasilnya digabungkan kembali menggunakan strategi agregasi sederhana (`aggregation_strategy="simple"`) untuk menyatukan sub-kata yang terpecah selama proses tokenisasi menjadi satu entitas utuh. Dengan pendekatan ini, tidak ada informasi lokasi atau nama tokoh yang hilang meskipun artikel berita tersebut sangat panjang.

```

# 6. Eksekusi Ekstraksi NER
print("Mulai mengekstrak entitas NER pada kolom 'Konten'...")
# Catatan: Karena menggunakan deep learning, proses ini mungkin memakan waktu.
# GPU (T4) yang kamu aktifkan sebelumnya akan sangat membantu mempercepat proses ini.
df_berita['entities'] = df_berita['Konten'].progress_apply(extract_entities_chunked)

# 7. Format Menjadi Dataframe Detail untuk Analisis
print("Menyusun ulang format data...")
rows = []
for idx, row in df_berita.iterrows():
    if isinstance(row['entities'], list):
        for text, ent_type in row['entities']:
            rows.append({
                'id_berita': idx,
                'entitas': text,
                'label': ent_type
            })

df_ner_detail = pd.DataFrame(rows)

# 8. Simpan Hasil ke CSV Baru
print("Menyimpan hasil ekstraksi NER...")
df_ner_detail.to_csv('hasil_ner_berita.csv', index=False)

print("\nProses selesai! File 'hasil_ner_berita.csv' sudah tersimpan.")
display(df_ner_detail.head(10))

```

2.2 Analisis Temuan dan Output

Berdasarkan hasil pemrosesan yang terekam dalam file hasil_ner_berita.csv, sistem berhasil mengekstraksi ribuan entitas yang dikelompokkan ke dalam beberapa kategori utama. Temuan spesifik dari data tersebut meliputi:

- Entitas Geopolitik (GPE) dan Lokasi (LOC): Ini merupakan temuan paling krusial dalam konteks bencana. Model secara akurat mengidentifikasi wilayah administratif mulai dari skala makro seperti 'Kota Makassar' hingga skala mikro seperti 'Kecamatan Biringkanaya', 'Kelurahan Katimbang', dan 'Paccerakkang'. Selain itu, entitas lokasi (LOC) berhasil menangkap titik-titik penting yang menjadi pusat aktivitas warga atau evakuasi, seperti 'Masjid Nurul Ikhlas', 'SD Paccerakkang', dan 'Kantor Balai Kota Makassar'.
- Entitas Organisasi (ORG): Temuan pada kategori ini menunjukkan aktor-aktor institusional yang dominan dalam narasi berita. Nama lembaga seperti 'BMKG' (Badan Meteorologi, Klimatologi, dan Geofisika) dan 'BPBD Makassar' (Badan Penanggulangan Bencana Daerah) muncul secara konsisten, yang menandakan bahwa berita-berita tersebut memiliki kredibilitas informasi karena merujuk pada otoritas teknis yang berwenang.
- Entitas Tokoh (PER) dan Waktu (DAT): Model berhasil menangkap nama individu yang memberikan pernyataan atau terlibat dalam aksi lapangan, salah satunya adalah 'Munafri Arifuddin'. Untuk entitas waktu, sistem mengekstrak tanggal kejadian secara spesifik seperti 'Senin (12/1/2026)' serta referensi waktu musiman seperti bulan 'Januari' dan 'Februari' yang sering dikaitkan dengan puncak curah hujan.

Adapun seluruh label yang digunakan adalah sebagai berikut:

- GPE (Geopolitical Entity)
- PER (Person)
- LOC (Location)
- ORG (Organization)
- NOR (Norm/Title/Role)
- DAT (Date)
- QTY (Quantity)
- CRD (Cardinal Number)
- EVT (Event)
- PRD (Product)
- TIM (Time)
- FAC (Facility)
- ORD (Ordinal Number)
- MON (Money)
- REG (Religion)
- LAW (Law)
- PRC (Percentage)
- WOA (Work of Art)
- LAN (Language)

Proses selesai! File 'hasil_ner_berita.csv' sudah tersimpan.

	id_berita	entitas	label
0	3	wali kota makassar	NOR
1	3	munafr 'appi 'arifuddin	PER
2	3	kelurahan katimbang	GPE
3	3	pacerakkang	GPE
4	3	kecamatan biringkanaya	GPE
5	3	kota makassar	GPE
6	3	app	PER
7	3	##i	ORG
8	3	kantor balai kota makassar	LOC
9	3	senin (12 / 1 / 2026)	DAT

2.3 Visualisasi Named Entity Recognition

Untuk mempermudah interpretasi data dalam jumlah besar, hasil ekstraksi NER disajikan melalui pendekatan visualisasi yang komprehensif. Visualisasi ini berfungsi sebagai jembatan bagi pembaca untuk melihat pola sebaran entitas tanpa harus memeriksa ribuan baris data CSV secara manual.

```
# Load file hasil NER
df_ner = pd.read_csv('hasil_ner_berita.csv')

id_berita_pilihan = 3

# Filter data hanya untuk ID berita tersebut
df_sample = df_ner[df_ner['id_berita'] == id_berita_pilihan]

if df_sample.empty:
    html_ner = f"<p style='color: white;'>Tidak ada entitas yang ditemukan pada ID berita {id_berita_pilihan}."
else:
    # Ubah format kolom dataframe menjadi list of tuples
    ner_tuples = list(zip(df_sample['entitas'], df_sample['label']))

    # Buat HTML-nya
    html_ner = visualize_ner_tags_highlight(ner_tuples, max_tokens=150)

# Tampilkan ke layar
display(HTML(f"""
<h3>Visualisasi Hasil NER (Berita ID: {id_berita_pilihan})</h3>
<div style="line-height:2.1; font-size:15px; padding:15px; border-radius:8px;">
{html_ner}
</div>
"""))
```

Salah satu bentuk visualisasi utama yang dihasilkan adalah pemetaan entitas langsung di atas naskah berita menggunakan teknik highlighting. Dalam visualisasi ini, setiap kata yang teridentifikasi sebagai entitas akan diberi warna latar belakang yang berbeda sesuai dengan kategorinya—misalnya, warna biru untuk lokasi (LOC), hijau untuk organisasi (ORG), dan merah untuk tokoh (PER). Pendekatan visual ini sangat efektif untuk memverifikasi secara cepat apakah model telah menangkap konteks kalimat dengan benar dalam sebuah artikel utuh.

Visualisasi Hasil NER (Berita ID: 3)



3. N-Grams

3.1 Alur Logika dan Prapemrosesan

Landasan utama dari analisis N-Grams dalam kode ini adalah penggunaan data dari kolom After_lemma. Tahap lemmatization ini merupakan langkah yang sangat vital; dengan mengubah kata-kata seperti "mengungsi", "pengungsian", dan "diungsikan" menjadi kata dasar "ungsi", sistem dapat menghitung frekuensi topik secara objektif tanpa terdistraksi oleh variasi imbuhan.

```
# Download resource pendukung
nltk.download('punkt')

def get_top_ngrams(corpus, n=None, top_k=15):
    """Fungsi untuk mengekstrak n-grams dari kolom After_lemma"""
    words = " ".join(corpus.astype(str)).split()
    n_grams = ngrams(words, n)
    counts = Counter(n_grams)
    return counts.most_common(top_k)

def plot_ngrams(ngram_data, title, color_palette='magma'):
    """Fungsi untuk visualisasi grafik batang"""
    if not ngram_data:
        return
    labels = [" ".join(item[0]) for item in ngram_data]
    counts = [item[1] for item in ngram_data]

    plt.figure(figsize=(10, 7))
    sns.barplot(x=counts, y=labels, hue=labels, palette=color_palette, legend=False)
    plt.title(title, fontsize=15, fontweight='bold')
    plt.xlabel('Frekuensi Muncul')
    plt.ylabel('Kata/Frasa')
    plt.grid(axis='x', linestyle='--', alpha=0.6)
    plt.show()
```

Proses komputasi dilakukan dengan mengekstraksi Unigrams untuk melihat kata tunggal yang paling berpengaruh dan Bigrams untuk menangkap konteks gabungan kata yang lebih bermakna (seperti "curah hujan"). Frekuensi kemunculan ini kemudian dihitung menggunakan fungsi Counter dan divisualisasikan melalui grafik batang

```
print("Memproses Unigrams...")

# Visualisasi untuk Banjir Biasa menggunakan kolom After_lemma
top_1_biasa = get_top_ngrams(df_sentimen_banjir['After_lemma'], n=1, top_k=20)
plot_ngrams(top_1_biasa, "Top 20 Unigrams: Banjir Biasa", 'magma')

# Visualisasi untuk Banjir Bandang menggunakan kolom After_lemma
top_1_bandang = get_top_ngrams(df_sentimen_bandang['After_lemma'], n=1, top_k=20)
plot_ngrams(top_1_bandang, "Top 20 Unigrams: Banjir Bandang", 'viridis')

print("Memproses Bigrams...")

# Visualisasi untuk Banjir Biasa[cite: 1]
top_2_biasa = get_top_ngrams(df_sentimen_banjir['After_lemma'], n=2, top_k=15)
plot_ngrams(top_2_biasa, "Top 15 Bigrams: Banjir Biasa", 'rocket')

# Visualisasi untuk Banjir Bandang[cite: 1]
top_2_bandang = get_top_ngrams(df_sentimen_bandang['After_lemma'], n=2, top_k=15)
plot_ngrams(top_2_bandang, "Top 15 Bigrams: Banjir Bandang", 'flare')

print("Memproses Trigrams...")

# Visualisasi untuk Banjir Biasa[cite: 1]
top_3_biasa = get_top_ngrams(df_sentimen_banjir['After_lemma'], n=3, top_k=10)
plot_ngrams(top_3_biasa, "Top 10 Trigrams: Banjir Biasa", 'mako')

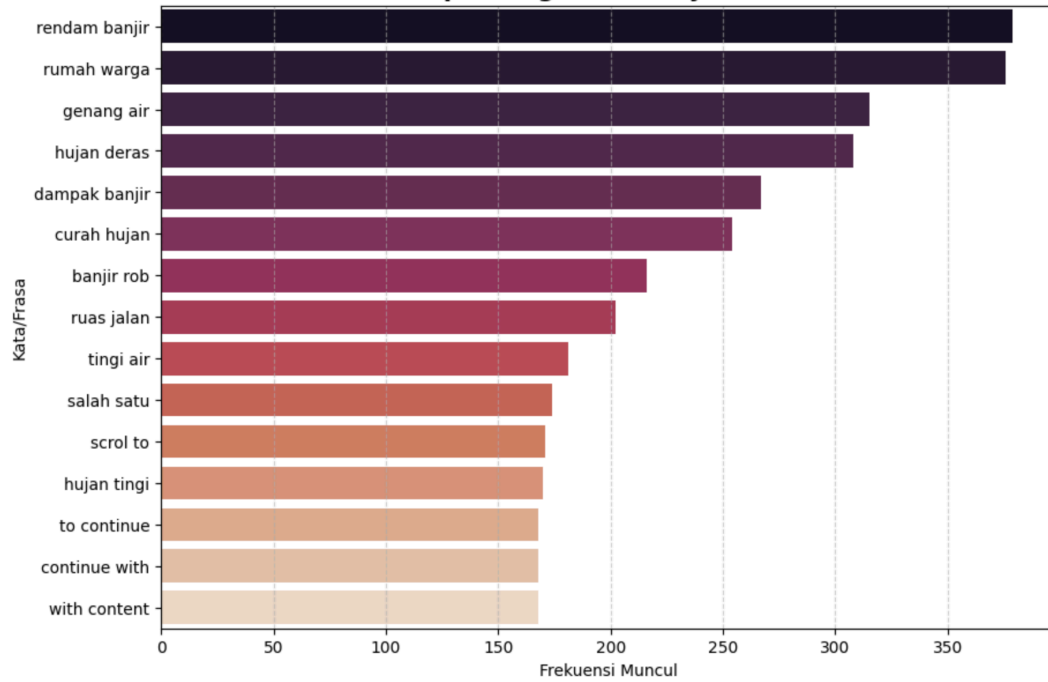
# Visualisasi untuk Banjir Bandang[cite: 1]
top_3_bandang = get_top_ngrams(df_sentimen_bandang['After_lemma'], n=3, top_k=10)
plot_ngrams(top_3_bandang, "Top 10 Trigrams: Banjir Bandang", 'crest')
```

3.2 Perbandingan Narasi: Banjir Biasa vs. Banjir Bandang

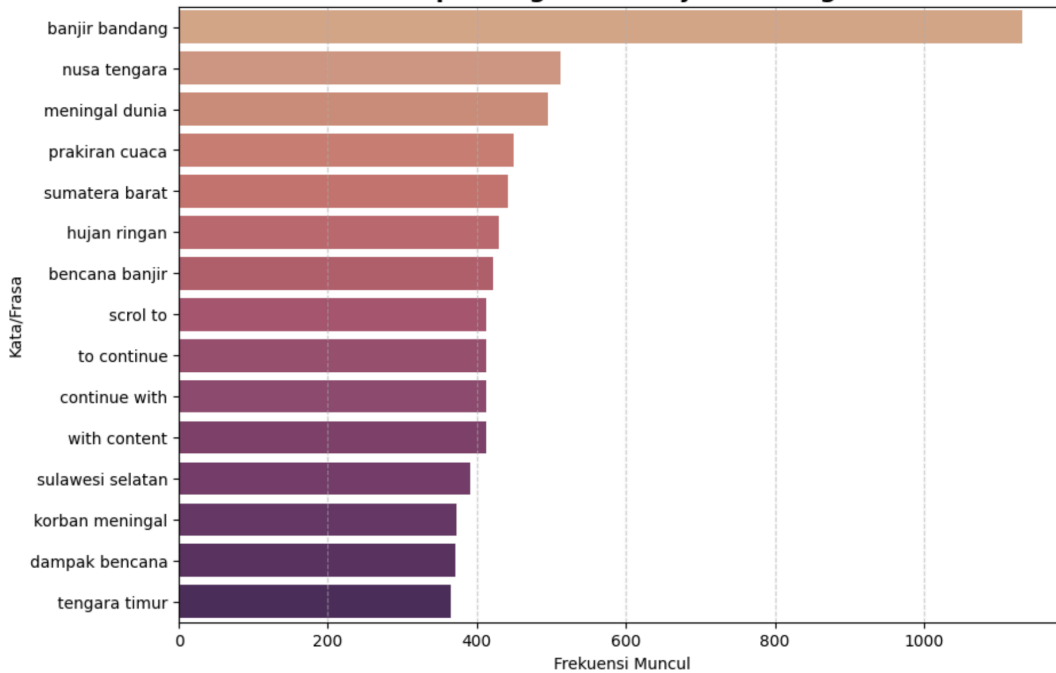
Visualisasi yang dihasilkan melalui fungsi `plot_ngrams` mengungkapkan perbedaan framing yang sangat kontras antara dua jenis bencana ini:

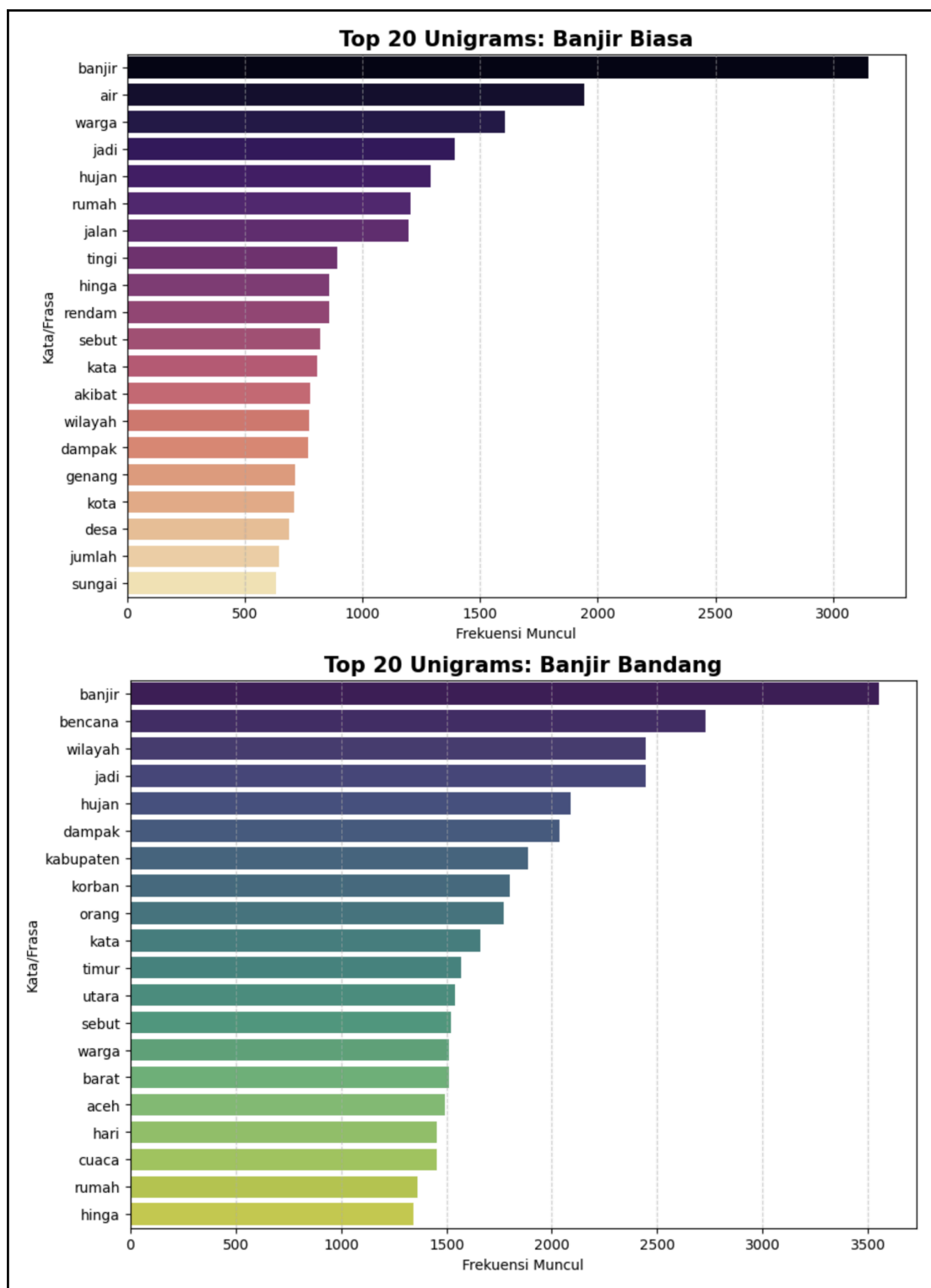
- Narasi Banjir Biasa: Didominasi oleh kata-kata seperti 'makassar', 'warga', 'hujan', dan 'siaga'. Hal ini menunjukkan bahwa pemberitaan banjir biasa cenderung bersifat informatif-antisipatif, berfokus pada kondisi cuaca harian dan kesiapsiagaan warga kota terhadap genangan air yang mungkin muncul.
- Narasi Banjir Bandang: Secara drastis bergeser ke arah deskripsi kerusakan fisik dengan kata kunci utama seperti 'rumah', 'rusak', 'korban', dan 'daerah'. Media memberikan penekanan yang jauh lebih besar pada aspek tragedi, kerugian materiil, dan dampak kemanusiaan yang ditimbulkan oleh terjangan air bah.

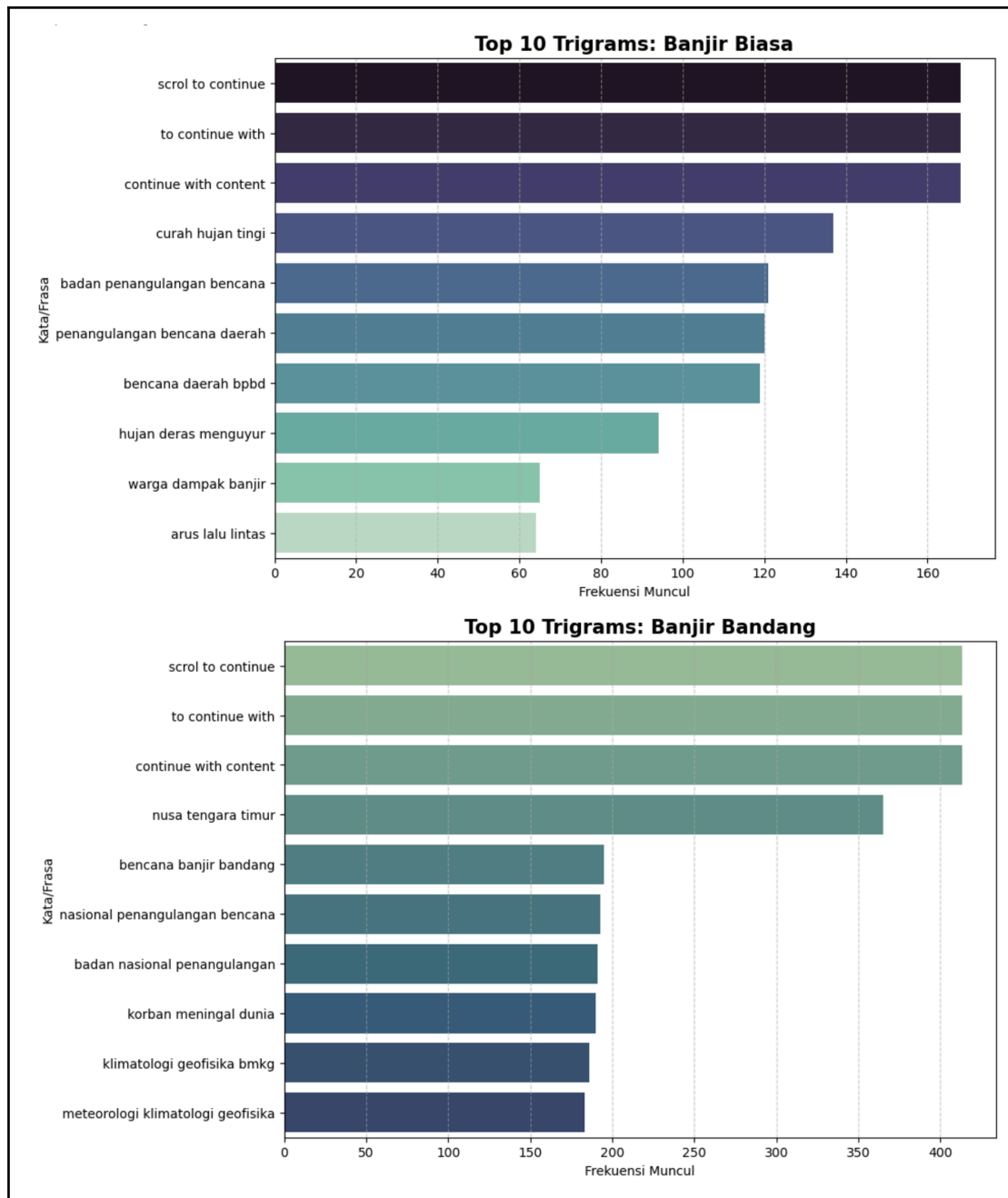
Top 15 Bigrams: Banjir Biasa



Top 15 Bigrams: Banjir Bandang







4. TF-IDF (Vectorization)

4.1. Persiapan Data

Pada tahap ini dilakukan proses pemuatan dan persiapan data yang akan digunakan dalam analisis. Data dibaca menggunakan library pandas dari file yang telah melalui tahap preprocessing, khususnya pada kolom *After_lemma* yang berisi teks hasil normalisasi. Selanjutnya, dilakukan pembersihan data dengan menghapus

nilai kosong (missing values) serta melakukan standarisasi pada kolom Sentimen menjadi huruf kecil untuk memastikan konsistensi data.

Data kemudian dipisahkan ke dalam dua kategori, yaitu banjir biasa dan banjir bandang, berdasarkan label sentimen yang dimiliki. Hasil pemisahan menunjukkan bahwa terdapat 683 dokumen pada kategori banjir biasa dan 971 dokumen pada kategori banjir bandang. Kedua kategori ini kemudian digabungkan kembali untuk keperluan proses vectorization.

```
# Load data
df = pd.read_csv('data_banjir_master_lemmatized.csv', usecols=['After_lemma', 'Sentimen'])
df = df.dropna(subset=['After_lemma'])
df['Sentimen'] = df['Sentimen'].str.strip().str.lower()

# Pisah per kategori
corpus_biasa = df[df['Sentimen'] == 'banjir']['After_lemma'].astype(str).tolist()
corpus_bandang = df[df['Sentimen'] == 'bandang']['After_lemma'].astype(str).tolist()
corpus_all = corpus_biasa + corpus_bandang
labels_all = ['Banjir Biasa'] * len(corpus_biasa) + ['Banjir Bandang'] * len(corpus_bandang)

print(f"Jumlah dokumen Banjir Biasa : {len(corpus_biasa)}")
print(f"Jumlah dokumen Banjir Bandang: {len(corpus_bandang)}")

Jumlah dokumen Banjir Biasa : 683
Jumlah dokumen Banjir Bandang: 971
```

4.2. Proses TF-IDF (Vectorization)

Pada tahap ini dilakukan proses pembobotan kata menggunakan metode TF-IDF (Term Frequency–Inverse Document Frequency). Proses ini bertujuan untuk mengubah data teks menjadi representasi numerik serta mengukur tingkat kepentingan suatu kata dalam dokumen terhadap keseluruhan korpus. Proses TF-IDF dilakukan menggunakan *TfidfVectorizer* dari library *scikit-learn* dengan parameter *max_features* sebesar 500, *min_df* sebesar 2, *max_df* sebesar 0.95, serta *ngram_range* (1,2) untuk mempertimbangkan unigram dan bigram.

Pembatasan fitur dilakukan untuk mengurangi dimensi data serta menghindari penggunaan kata-kata yang kurang relevan. Parameter *min_df* digunakan untuk menghilangkan kata yang terlalu jarang muncul, sedangkan *max_df* digunakan untuk mengabaikan kata yang terlalu sering muncul di hampir seluruh dokumen. Penggunaan unigram dan bigram juga memungkinkan sistem menangkap konteks kata secara lebih spesifik. Hasil dari proses ini berupa matriks TF-IDF dengan ukuran (1654, 500), yang menunjukkan jumlah dokumen dan jumlah fitur (kata) yang digunakan.

4.3. Pembentukan Data TF-IDF

Setelah proses vectorization dilakukan, hasil matriks TF-IDF kemudian dikonversi ke dalam bentuk *DataFrame* untuk mempermudah analisis. Setiap baris merepresentasikan dokumen, sedangkan setiap kolom merepresentasikan fitur kata dengan nilai bobot TF-IDF.

Selain itu, ditambahkan kolom kategori untuk membedakan antara dokumen banjir biasa dan banjir bandang. Dengan struktur ini, analisis dapat dilakukan secara terpisah berdasarkan kategori masing-masing.

4.4. Analisis Kata Dominan

Pada tahap ini dilakukan analisis untuk mengidentifikasi kata-kata yang memiliki nilai TF-IDF tertinggi pada masing-masing kategori. Proses ini dilakukan dengan menghitung rata-rata bobot TF-IDF setiap kata dalam satu kategori, kemudian diurutkan untuk mendapatkan kata-kata paling dominan.

Hasil analisis menunjukkan bahwa pada kategori banjir biasa, kata dengan skor tertinggi adalah *banjir* (0.129), diikuti oleh *air* (0.116), *jalan* (0.093), *warga* (0.089), dan *genang* (0.078). Kata-kata ini mencerminkan karakteristik banjir biasa yang umumnya berkaitan dengan genangan air di jalan dan aktivitas masyarakat terdampak. Kemunculan kata seperti *bpbd*, *jakarta*, dan *genang air* juga mengindikasikan bahwa pemberitaan banjir biasa lebih banyak berfokus pada wilayah perkotaan dan respons pemerintah daerah.

Sementara itu, pada kategori banjir bandang, kata dominan adalah *banjir* (0.068), *bencana* (0.065), *korban* (0.061), *orang* (0.057), dan *aceh* (0.055). Berbeda dengan banjir biasa, kemunculan kata seperti *korban*, *bencana*, *meninggal*, dan bigram *banjir bandang* mencerminkan tingkat keparahan yang jauh lebih tinggi. Kata *aceh*, *sumatera*, dan *kabupaten* mengindikasikan bahwa berita banjir bandang lebih banyak terjadi di luar wilayah Jawa.

```
# Top TF-IDF per kategori
def get_top_tfidf(df_tfidf, kategori, top_n=20):
    df_cat = df_tfidf[df_tfidf['kategori'] == kategori].drop(columns='kategori')
    mean_score = df_cat.mean(axis=0).sort_values(ascending=False)
    return mean_score.head(top_n).reset_index().rename(columns={'index': 'Kata', 0: 'TF-IDF Score'})

top_biasa = get_top_tfidf(df_tfidf, 'Banjir Biasa', top_n=20)
top_bandang = get_top_tfidf(df_tfidf, 'Banjir Bandang', top_n=20)

top_biasa.columns = ['Kata', 'TF-IDF Score']
top_bandang.columns = ['Kata', 'TF-IDF Score']

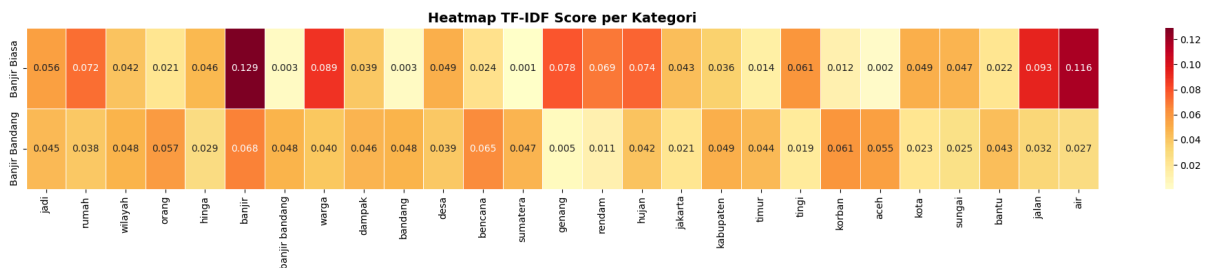
print("\n=== TOP 20 KATA - BANJIR BIASA ===")
display(top_biasa)
print("\n=== TOP 20 KATA - BANJIR BANDANG ===")
display(top_bandang)
```

=== TOP 20 KATA - BANJIR BIASA ===			=== TOP 20 KATA - BANJIR BANDANG ===		
	Kata	TF-IDF Score		Kata	TF-IDF Score
0	banjir	0.128760	0	banjir	0.067598
1	air	0.116469	1	bencana	0.065303
2	jalan	0.092837	2	korban	0.061039
3	warga	0.088698	3	orang	0.057131
4	genang	0.077997	4	aceh	0.055375
5	hujan	0.073861	5	kabupaten	0.048833
6	rumah	0.071621	6	bandang	0.048196
7	rendam	0.069410	7	wilayah	0.047645
8	tingi	0.060973	8	banjir bandang	0.047618
9	jadi	0.056372	9	sumatera	0.047134
10	desa	0.049355	10	dampak	0.046479
11	kota	0.049259	11	jadi	0.045372
12	sungai	0.047014	12	timur	0.044497
13	hinga	0.046268	13	bantu	0.042882
14	jakarta	0.043314	14	hujan	0.041719
15	bpbd	0.042925	15	kata	0.041077
16	sebut	0.042392	16	warga	0.039925
17	akibat	0.042327	17	meninggal	0.039661
18	wilayah	0.041638	18	desa	0.038675
19	genang air	0.040483	19	rumah	0.038378

4.5. Visualisasi TF-IDF Menggunakan Heatmap

Heatmap disusun dengan dua baris yang merepresentasikan kategori banjir biasa dan banjir bandang, serta kolom yang menunjukkan kata-kata yang dianalisis. Intensitas warna menunjukkan besar kecilnya nilai TF-IDF, di mana warna yang lebih gelap mengindikasikan nilai yang lebih tinggi.

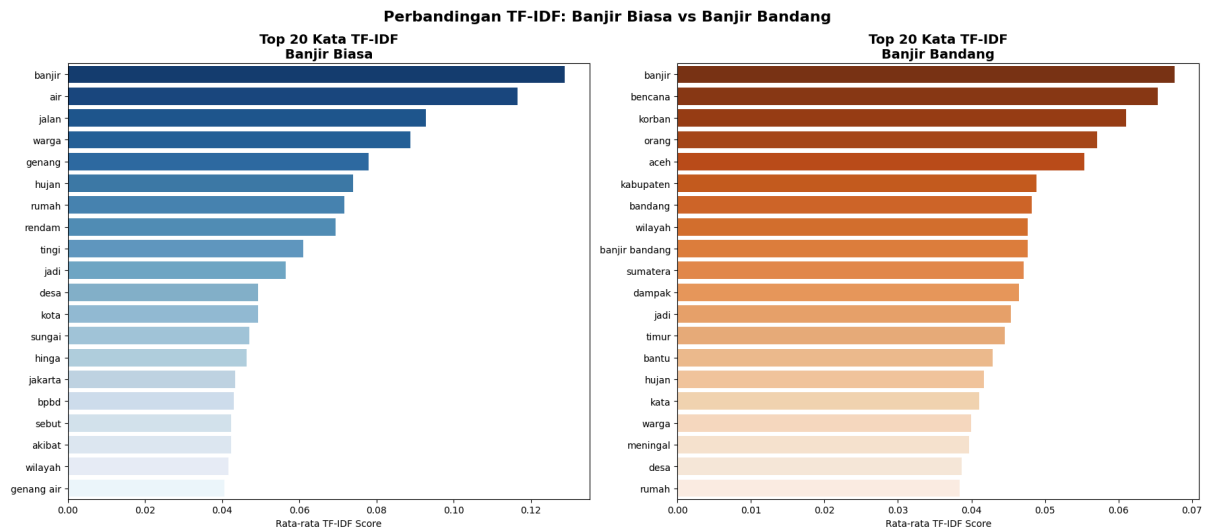
Berdasarkan visualisasi yang dihasilkan, terlihat bahwa beberapa kata memiliki distribusi nilai yang berbeda secara signifikan antara kedua kategori. Hal ini menunjukkan adanya perbedaan pola penggunaan kata antara banjir biasa dan banjir bandang.



4.6. Visualisasi Perbandingan Kata (Bar Chart)

Selain menggunakan heatmap, dilakukan juga visualisasi dalam bentuk bar chart untuk menampilkan kata-kata dengan nilai TF-IDF tertinggi pada masing-masing kategori.

Pada kategori banjir biasa, kata-kata dominan seperti *banjir*, *air*, *jalan*, dan *warga* mencerminkan kondisi lingkungan serta aktivitas masyarakat terdampak. Sementara itu, pada kategori banjir bandang, kata-kata seperti *bencana*, *korban*, *bandang*, dan *dampak* menunjukkan fokus pada tingkat keparahan kejadian. Visualisasi ini memperjelas perbedaan karakteristik antara kedua kategori serta memperkuat hasil analisis sebelumnya.



4.7. Penyimpanan Hasil

Pada tahap akhir, hasil berupa daftar kata dengan skor TF-IDF serta matriks TF-IDF disimpan dalam format `.csv` untuk memudahkan penggunaan kembali data. Matriks TF-IDF yang dihasilkan kemudian digunakan sebagai input pada tahap klasifikasi sentimen, di mana setiap dokumen direpresentasikan dalam bentuk numerik agar dapat diproses oleh algoritma machine learning.

```
# Simpan hasil
top_biasa['kategori'] = 'Banjir Biasa'
top_bandang['kategori'] = 'Banjir Bandang'

df_top_tfidf = pd.concat([top_biasa, top_bandang], ignore_index=True)
df_top_tfidf.to_csv('tfidf_top_kata.csv', index=False)
df_tfidf.to_csv('tfidf_matrix.csv', index=False)

print("✅ File 'tfidf_top_kata.csv' dan 'tfidf_matrix.csv' berhasil disimpan!")

✅ File 'tfidf_top_kata.csv' dan 'tfidf_matrix.csv' berhasil disimpan!
```

5. Sentiment Analysis

Pada penelitian ini, klasifikasi sentimen berita banjir difokuskan untuk membedakan dua kategori utama: **Banjir Biasa** (banjir temporer, genangan) dan **Banjir Bandang** (banjir ekstrem, kerusakan berat, korban jiwa). Untuk tujuan ini, dua pendekatan klasifikasi dievaluasi dan dibandingkan: Keyword-based Classification dan Machine Learning Classification menggunakan algoritma Naive Bayes.

5.1. Evaluasi Klasifikasi Berbasis Kata Kunci (Keyword-based)

Pendekatan pertama menggunakan metode klasifikasi berbasis kata kunci (Keyword-based). Metode ini beroperasi tanpa proses training dan hanya mengandalkan pencocokan kata-kata kunci (keywords) yang secara manual ditetapkan sebagai representasi dari setiap kategori banjir.

- **Kata Kunci Banjir Bandang (Prioritas):** 'banjir bandang', 'bandang', 'terseret', 'menghanyutkan', 'menerjang', 'meninggal', 'korban jiwa', 'tewas', 'longsor', 'hancur', 'rusak berat', 'roboh', 'tertimbun', 'material', 'lumpur'.
- **Kata Kunci Banjir Biasa:** 'terendam', 'genangan', 'mengungsi', 'pengungsi', 'surut', 'meluap', 'rob', 'drainase', 'ketinggian air', 'banjir kiriman', 'tergenang', 'macet', 'lumpuh'.

Algoritma menghitung skor kemunculan kata kunci dari kedua daftar tersebut dalam setiap artikel. Jika skor kata kunci "bandang" lebih besar, maka artikel diklasifikasikan sebagai "Banjir Bandang", dan sebaliknya. Jika terdapat skor yang sama, prioritas diberikan pada kategori "Banjir Bandang" mengingat pentingnya deteksi dini pada jenis bencana ekstrem ini.

```
=== EVALUASI KEYWORD-BASED ===
Artikel terevaluasi : 1461 dari 1571
Tidak terklasifikasi: 110
Akurasi              : 84.05%

Classification Report:

```

	precision	recall	f1-score	support
Banjir Bandang	0.88	0.83	0.85	815
Banjir Biasa	0.80	0.85	0.83	646
accuracy			0.84	1461
macro avg	0.84	0.84	0.84	1461
weighted avg	0.84	0.84	0.84	1461

Berdasarkan hasil pengujian Keyword-based terhadap 1.571 artikel berita, terdapat 110 artikel yang masuk ke dalam kategori "Tidak Diketahui" karena tidak memuat satupun kata kunci dari kedua daftar yang telah dibuat. Dari sisa 1.461 artikel yang berhasil dievaluasi, metode ini mencapai akurasi sebesar **84.05%**. Hal ini menunjukkan bahwa definisi kata kunci manual cukup mampu menangkap pola dasar, namun kelemahannya terletak pada recall dan ketidakmampuannya memproses dokumen yang menggunakan variasi kosakata di luar kamus buatan peneliti.

Confusion Matrix — Keyword-based

Aktual	Banjir Biasa	552	94
	Banjir Bandang	139	676
		Banjir Biasa	Banjir Bandang
		Prediksi	

Confusion Matrix di atas memperlihatkan distribusi tebakan benar dan salah dari metode Keyword-based. Terlihat bahwa masih terdapat sejumlah besar prediksi False Positive dan False Negative. Hal ini mengindikasikan bahwa beberapa berita banjir biasa menggunakan kosakata yang tumpang tindih dengan banjir bandang (atau sebaliknya), sehingga metode berbasis aturan kaku mengalami kebingungan dalam memutuskannya.

5.2. Evaluasi Klasifikasi Machine Learning (Naive Bayes)

Pendekatan kedua menggunakan algoritma Machine Learning, yaitu **Multinomial Naive Bayes**. Berbeda dengan pendekatan berbasis aturan, model Naive Bayes dilatih (training) untuk mengenali probabilitas kemunculan fitur kata berdasarkan kategori sentimennya.

Model ini menggunakan representasi numerik teks dalam bentuk **Matriks TF-IDF** sebagai fitur input. Matriks ini terdiri dari 1.654 dokumen (setelah penggabungan beberapa korpus tambahan) dengan 500 fitur kata (unigram dan bigram). Dataset dibagi dengan proporsi 80% untuk data latih (training data) dan 20% untuk data uji (testing data).

```

=== EVALUASI NAIVE BAYES ===
Akurasi: 93.05%

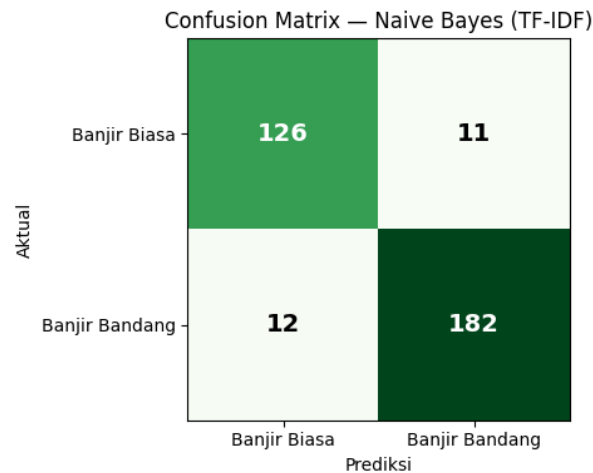
Classification Report:

```

	precision	recall	f1-score	support
Banjir Bandang	0.94	0.94	0.94	194
Banjir Biasa	0.91	0.92	0.92	137
accuracy			0.93	331
macro avg	0.93	0.93	0.93	331
weighted avg	0.93	0.93	0.93	331

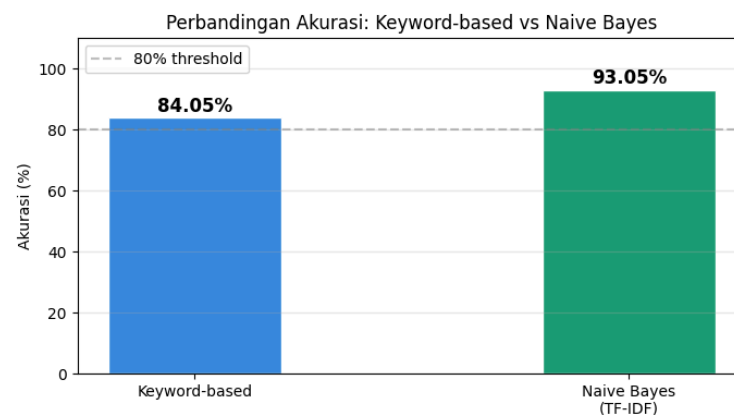
Kinerja model Naive Bayes menunjukkan peningkatan yang sangat drastis. Model mampu memprediksi data uji dengan tingkat akurasi mencapai **93.05%**. Nilai presisi untuk kategori Banjir Bandang mencapai 0.94 dan Banjir Biasa mencapai 0.91.

Hal ini membuktikan bahwa algoritma Machine Learning mampu memahami konstelasi kalimat secara holistik dari kombinasi 500 kata fitur, tanpa ada satupun dokumen yang gagal terklasifikasi.



Melalui visualisasi Confusion Matrix untuk Naive Bayes, terlihat konsentrasi angka yang sangat pekat pada garis diagonal utama (True Positives). Jumlah kesalahan prediksi sangat minim. Model berhasil meminimalkan bias dan sukses memisahkan batas linguistik antara teks yang mendeskripsikan genangan air biasa dan teks yang melaporkan kerusakan akibat bandang.

5.3. Perbandingan dan Ringkasan Klasifikasi Sentimen

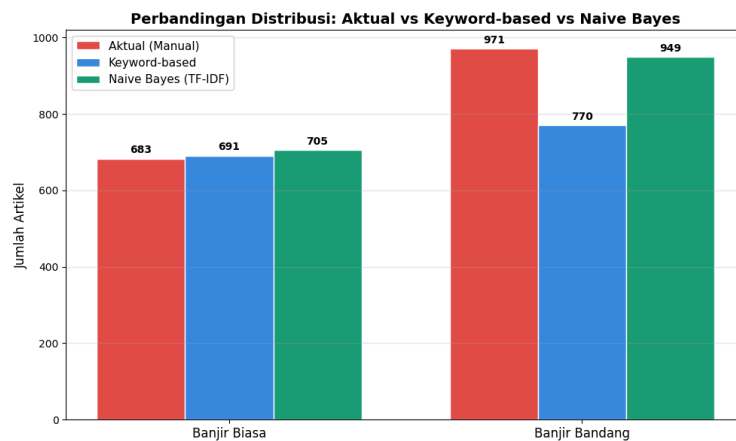


Perbandingan hasil kedua metode klasifikasi memperlihatkan keunggulan pendekatan Machine Learning dibandingkan metode berbasis aturan (rule-based).

Metode Klasifikasi	Akurasi
Keyword-based Classification	84.05%
Naive Bayes (TF-IDF) Classification	93.05%

Berdasarkan hasil pengujian, algoritma Naive Bayes terbukti lebih unggul dengan selisih akurasi sebesar **9,00%** dibandingkan metode Keyword-based. Hasil ini mengonfirmasi bahwa pendekatan machine learning yang mengandalkan representasi fitur TF-IDF jauh lebih efektif dalam menangkap pola linguistik yang kompleks, frekuensi kemunculan, dan hubungan semantik antar kata dalam teks berita banjir.

Sementara metode Keyword-based gagal mengklasifikasikan 110 artikel karena ketiadaan kata kunci yang cocok secara eksak (kaku), model Naive Bayes mampu menggeneralisasi probabilitas dari kombinasi 500 fitur kata (unigram dan bigram) sehingga dapat memberikan prediksi pada seluruh sampel data tanpa ada dokumen yang berstatus "Tidak Diketahui".



Grafik distribusi di atas membandingkan sebaran jumlah artikel berdasarkan data Aktual pelabelan manual, tebakan Keyword-based, dan tebakan Naive Bayes untuk seluruh data (1.654 dokumen). Terlihat bahwa batang histogram prediksi Naive Bayes (warna hijau) memiliki tinggi yang sangat menyerupai batang Aktual (warna merah) pada kedua kategori. Di sisi lain, metode Keyword-based (warna biru) memiliki selisih jumlah yang cukup signifikan karena adanya ratusan dokumen (110 artikel) yang gagal diklasifikasikan dan masuk ke status "Tidak Diketahui". Hal ini secara empiris membuktikan bahwa penggunaan representasi vektor TF-IDF dikombinasikan dengan Naive Bayes merupakan pendekatan paling handal dan robust untuk klasifikasi sentimen berita bencana alam berbahasa Indonesia.

BAB V

PENUTUP

1. Kesimpulan

Penelitian ini berhasil menerapkan pendekatan Natural Language Processing (NLP) secara komprehensif untuk menganalisis berita banjir berbahasa Indonesia. Melalui proses web scraping menggunakan library *newspaper3k* dan *BeautifulSoup* dari 230 sumber media daring, berhasil dikumpulkan sebanyak 2.498 berita banjir yang representatif sebagai basis dataset penelitian. Keberhasilan pengumpulan data ini menjadi pondasi penting bagi seluruh tahapan analisis berikutnya.

Pada tahap analisis linguistik, metode POS Tagging berbasis Stanza dan Named Entity Recognition menggunakan model IndoBERT terbukti efektif dalam mengidentifikasi pola distribusi kata serta mengekstraksi entitas penting seperti lokasi, aktor, dan keterangan waktu secara otomatis. Temuan ini menunjukkan bahwa berita banjir bandang cenderung didominasi kata kerja destruktif dan kata sifat berkonotasi negatif, sementara berita banjir biasa lebih banyak memuat kata benda yang merujuk pada kondisi lingkungan sehari-hari. Perbedaan pola ini mencerminkan karakteristik semantik yang khas dari masing-masing kategori berita.

Secara keseluruhan, kombinasi teknik N-Grams dan TF-IDF berhasil membuktikan bahwa representasi vektor dengan unigram dan bigram mampu menangkap perbedaan karakteristik linguistik yang signifikan antara berita banjir biasa dan banjir bandang secara terukur. Kata-kata seperti *bencana*, *korban*, dan *dampak* secara konsisten muncul dominan pada kategori banjir bandang, sedangkan kata seperti *jalan* dan *warga* mendominasi kategori banjir biasa. Hal ini membuktikan bahwa pendekatan NLP merupakan solusi yang efektif dan efisien untuk analisis berita bencana berbahasa Indonesia dalam skala besar.

2. Rekomendasi

Dari total 2.454 artikel yang dicoba, hanya 1.654 yang berhasil diekstrak karena berbagai kendala teknis seperti paywall, penghapusan artikel, dan pemblokiran server. Ke depannya, penelitian serupa sebaiknya mempertimbangkan penggunaan teknik scraping yang lebih beragam, misalnya dengan memanfaatkan API resmi portal berita, pengarsipan halaman secara berkala menggunakan Wayback Machine, atau rotasi user-agent dan proxy untuk mengurangi risiko pemblokiran. Dengan cakupan data yang lebih luas, representasi korpus akan lebih kuat dan hasil analisis menjadi lebih general.

Pada metode preprocessing dan klasifikasi meskipun Naive Bayes dengan TF-IDF sudah menunjukkan akurasi 93,05%, terdapat ruang untuk peningkatan lebih lanjut dengan mengeksplorasi model yang lebih kompleks seperti IndoBERT atau model transformer berbahasa Indonesia lainnya. Model berbasis konteks semacam itu mampu menangkap makna semantik yang lebih dalam dibandingkan representasi bag-of-words TF-IDF. Selain itu, augmentasi data pada kategori yang kurang terwakili dapat membantu mengurangi bias model terhadap satu kelas tertentu, terutama mengingat ketidakseimbangan jumlah antara 683 dokumen banjir biasa dan 971 dokumen banjir bandang.

DAFTAR PUSTAKA

- Khurana, D., Koli, A., Khatter, K., & Singh, S. (2025). Natural Language Processing: State of the Art, Current Trends and Challenges. *Artificial Intelligence Review*, 58(1). <https://doi.org/10.1007/s10462-025-11162-5>
- Russell, S. J., & Norvig, P. (2020). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson. <https://books.google.co.id/books?id=WklAEQAAQBAJ>
- Samuel, & Prasetya Kristiadi, D. (2025). Deteksi Teks Promosi Judi Online Menggunakan AI dengan Kombinasi NLP dan Deep Learning. *Jurnal Sistem Informasi dan Teknologi (SINTEK)*, 5(2), 179–185. <https://doi.org/10.56995/sintek.v5i2.239>
- Budi, I., & Suryono, R. R. (2023). Application of named entity recognition method for Indonesian datasets. Universitas Indonesia.
- Cavnar, W. B., & Trenkle, J. M. (1994). N-gram-based text categorization. Dalam *Proceedings of the 3rd Annual Symposium on Document Analysis and Information Retrieval* (hlm. 161–175). Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2018). BERT: Pre-training of deep bidirectional transformers for language understanding. arXiv preprint arXiv:1810.04805.
- Jurafsky, D., & Martin, J. H. (2026). *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition with Language Models* (3rd ed.). Online manuscript. <https://web.stanford.edu/~jurafsky/slp3/>